

Indian Institute of Technology Madras

BS Electronic Systems

**Signal Processing Project
Mid-Term Progress Report**

Project Title:	DSP-Based Motor Current Signature Analysis (MCSA) for Predictive Maintenance of Induction Motors
Project Code:	Custom (Own Topic Proposal)
Student Name:	Arpit Katiyar
Roll Number:	ES24F1100064
Team Members:	Solo
Submission Date:	30-08-2026

1 Project Overview

This project implements Motor Current Signature Analysis (MCSA): a signal-processing pipeline that detects developing mechanical faults in three-phase induction motors — such as worn bearings or cracked rotor bars — from the motor’s existing supply current alone, without adding any vibration sensors. A mechanical fault amplitude-modulates the stator current, producing small sidebands around the supply-frequency peak; the pipeline extracts and quantifies those sidebands using the FFT, Welch’s power spectral density estimate, and Hilbert-transform envelope analysis, and produces a health verdict. The system is built as an interactive, browser-based web application that walks through every stage of the analysis and is deployed publicly.

2 Work Completed So Far

1. **Dataset evaluation and selection:** Evaluated three candidate datasets and settled on a verified current-signature dataset (details in Section 7). This involved rejecting the originally-proposed MAFAULDA dataset (vibration/acoustic only, no current channel) and a second candidate (a slow trend log, not a raw waveform), and adopting the Figshare “Motor Fault Detection Data” dataset, which provides genuine raw current, voltage, and triaxial acceleration channels sampled at 50 kHz.
2. **Raw signal capture and file loading:** Implemented a streaming loader for large (~160–200 MB) recordings that reads the file in chunks rather than all at once, auto-detects a header row and multiple sensor channels, and prompts for the correct current column (I1/I2/I3) when a file has several columns — the chosen dataset’s files contain nine columns (three

acceleration axes, three current phases, three voltage phases). Both CSV and MATLAB (.mat) formats are supported.

3. **Bandpass filter (Butterworth, SOS form):** Implemented a fourth-order Butterworth bandpass filter in second-order-sections (SOS) form, applied with zero-phase forward-backward filtering. Verified on a synthetic sine wave that the output is bounded and stable (this replaced an earlier direct-form implementation that was numerically unstable — see Section 7.3).
4. **FFT spectral analysis (Hann-windowed):** Implemented a radix-2 FFT with a Hann window applied before transforming to suppress spectral leakage near the fundamental. Verified that a pure sinusoid reads back at the correct frequency and (window-corrected) amplitude.
5. **Welch PSD:** Implemented Welch’s averaged-periodogram power spectral density estimate (overlapping Hann-windowed segments) as a lower-variance cross-check against the single-shot FFT. Verified that its peak lands at the correct frequency.
6. **Hilbert-transform envelope analysis:** Implemented the analytic signal via the FFT and took its magnitude to obtain the envelope, then transformed the envelope to isolate the fault-induced modulation. Verified that the envelope of an amplitude-modulated carrier recovers the modulation frequency.
7. **Slip / sideband-frequency calculation:** Implemented the nameplate-based computation of synchronous speed, slip, slip frequency, and the first-order fault-sideband frequencies (Section 4.4), parameterised by supply frequency, rated RPM, and pole count.
8. **Feature extraction and rule-based verdict:** Implemented extraction of each sideband’s magnitude relative to the fundamental in decibels, and a configurable threshold rule (−40 dB default) producing a **HEALTHY / FAULT LIKELY** verdict. On synthetic signals with injected faults, the pipeline correctly separated a healthy case (~ -80 dB sideband) from a faulty case (~ -27 dB sideband).
9. **Automatic data-quality checks:** Built a trend-log detector (long runs of identical samples) and a fundamental-dominance check (warns when the entered supply frequency is not actually a dominant spectral peak), both prompted directly by real problems encountered during testing (Sections 7.2 and 7.5).
10. **User interface and deployment:** Built the two-tab interface (single-file pipeline walk-through and healthy-versus-faulty comparison) and deployed the application publicly.

3 Screenshots / Results So Far

The deployed application is shown below, walking through the full pipeline on a real recording from the chosen dataset. Figure 1 shows the instrument-panel interface with the parameter sidebar and the multi-column upload; the file’s nine columns are detected and the current phase I1 is selected as the signal to analyse.

Figure 2 shows the raw current waveform and the zero-phase Butterworth bandpass-filtered signal. The application automatically detected that each sample was repeated five times (constant oversampling) and corrected the effective sampling rate from the entered 50 kHz to the true 10 kHz.

Figure 3 shows two of the three spectral views — the Welch power spectral density and the Hilbert envelope spectrum — with the fundamental and predicted sideband markers overlaid.

Figure 4 shows the extracted-feature table, the health verdict, the automatic oversampling notice, and the threshold-sensitivity strip.

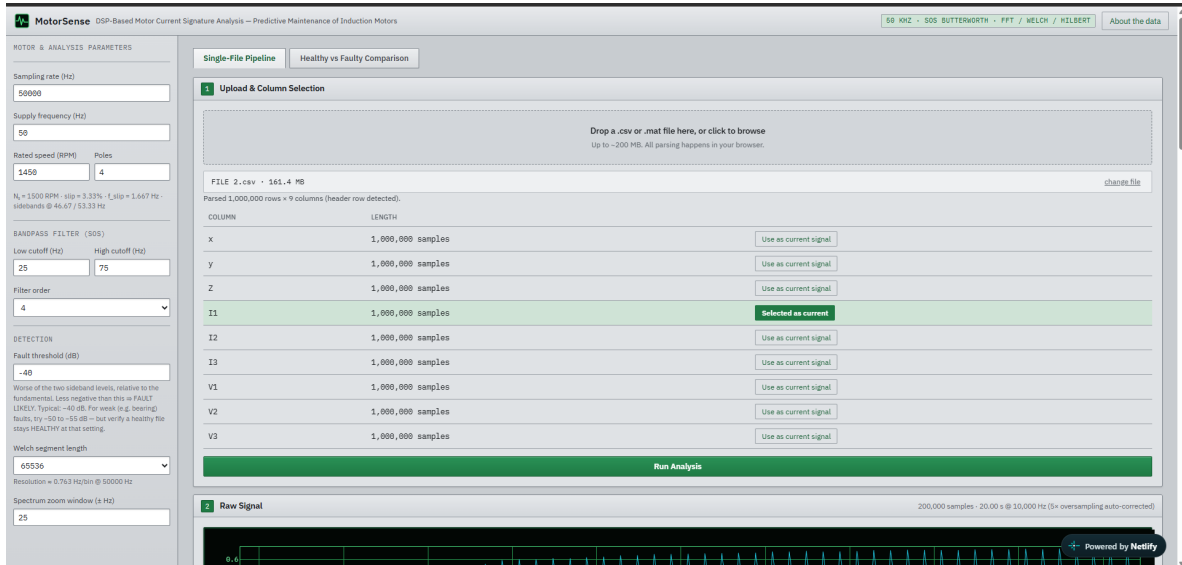


Figure 1: Deployed application (single-file pipeline tab): parameter sidebar, file upload, and current-column selection. The nameplate readout confirms $N_s = 1500$ RPM, slip 3.33%, $f_{\text{slip}} = 1.667$ Hz, and predicted sidebands at 46.67/53.33 Hz.



Figure 2: Raw current waveform (top) and the SOS Butterworth bandpass-filtered signal (bottom), with the $5\times$ oversampling automatically detected and corrected.

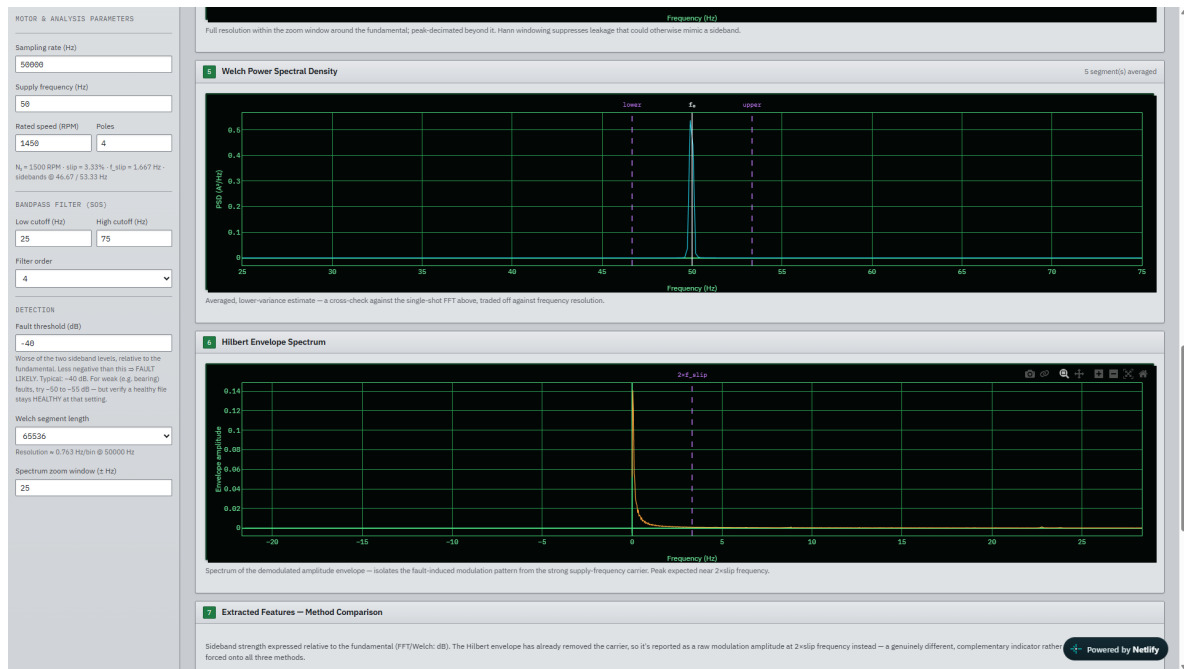


Figure 3: Welch PSD (top) and Hilbert envelope spectrum (bottom). The envelope spectrum's peak near $2f_{\text{slip}}$ is the fault-modulation indicator, separated from the strong carrier.

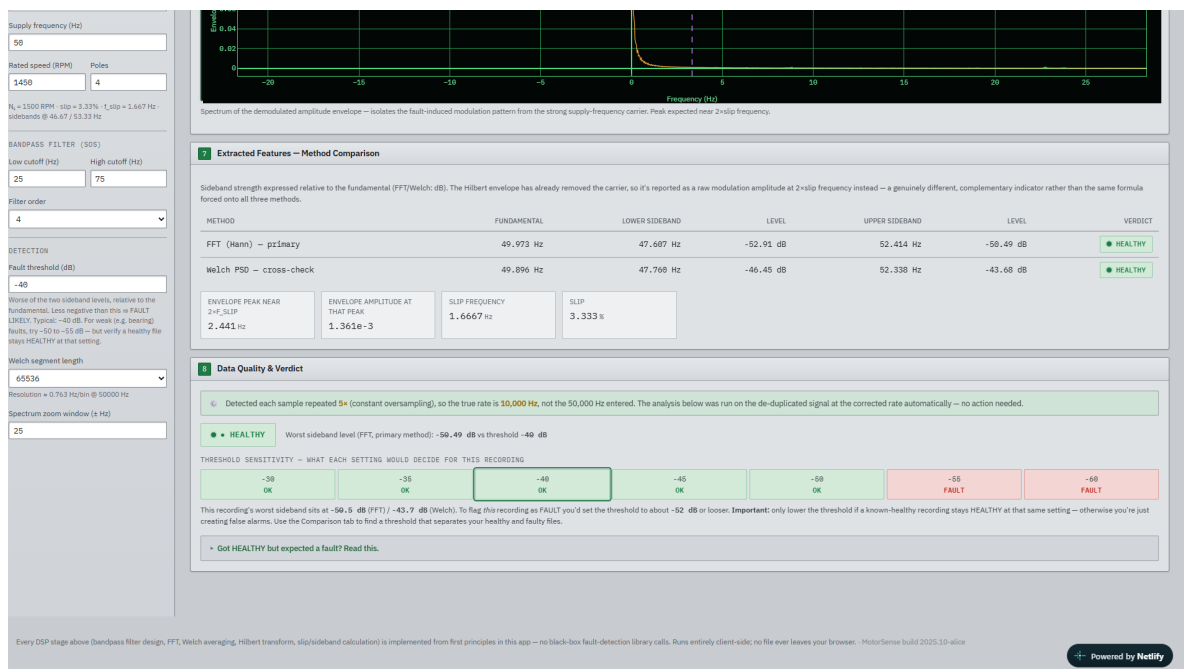


Figure 4: Extracted features, HEALTHY verdict, the automatic $5\times$ oversampling correction notice, and the threshold-sensitivity strip showing what verdict each candidate threshold would produce for this recording.

Method	Fundamental	Lower SB	Level	Upper SB	Level
FFT (Hann) — primary	49.973 Hz	47.607 Hz	−52.91 dB	52.414 Hz	−50.49 dB
Welch PSD — cross-check	49.896 Hz	47.760 Hz	−46.45 dB	52.338 Hz	−43.68 dB

Envelope peak near $2f_{\text{slip}}$	Envelope amplitude there	Slip frequency	Slip
2.441 Hz	1.361×10^{-3}	1.6667 Hz	3.333%

- Detected each sample repeated $5\times$ (constant oversampling), so the true rate is 10,000 Hz, not the 50,000 Hz entered. The analysis was run on the de-duplicated signal at the corrected rate automatically — no action needed.

• **HEALTHY** Worst sideband level (FFT, primary method): −50.49 dB vs threshold −40 dB

Threshold sensitivity — what each setting would decide for this recording

−30	−35	−40	−45	−50	−55	−60
OK	OK	OK	OK	OK	FAULT	FAULT

Table 1: Extracted results for a healthy recording: FFT and Welch sideband levels, envelope peak, slip, the oversampling-correction notice, the HEALTHY verdict, and the threshold-sensitivity row.

On synthetic test signals, where a fault sideband of known frequency and amplitude was deliberately injected into an otherwise clean simulated current waveform, the pipeline correctly separated the healthy case (sideband roughly 75–80 dB below the fundamental) from the faulty case (sideband roughly 27 dB below the fundamental) at the default threshold. On the real recording above, with the supply frequency set to the motor’s actual 50 Hz, the pipeline returned a healthy verdict with a worst sideband level of -50.49 dB (FFT) — consistent with the range expected for a healthy motor, and well below the -40 dB fault threshold.

4 Workflow / Approach

The pipeline maps each stage directly onto a specific course topic. The overall flow is shown in Figure 5.

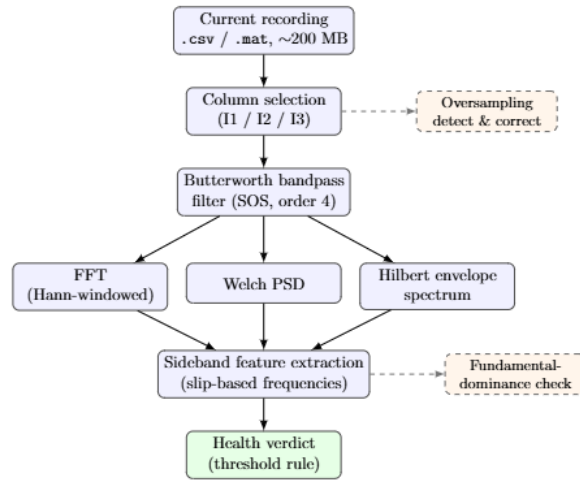


Figure 5: End-to-end pipeline: from the raw current recording through column selection (with automatic oversampling detection and correction), SOS bandpass filtering, the three parallel spectral methods, slip-based sideband feature extraction (with the fundamental-dominance check), to the rule-based health verdict.

Comparing the three spectral methods (FFT, Welch PSD, Hilbert envelope) against each other on the same recording — rather than committing to only one — was a deliberate design choice, and the proposal was reviewed favourably in part because it promised exactly this comparison.

5 Tools & Technologies Used

- **Language:** JavaScript (ES2017+), running entirely in the browser.
- **DSP implementation:** All signal-processing routines (radix-2 FFT, Hann window, Welch PSD, Hilbert transform, Butterworth SOS design and filtering, slip/sideband maths, feature extraction, verdict logic, and data-quality checks) written from first principles — no higher-level fault-detection library — so every stage stays traceable to the formulas in Section 4.4.
- **Supporting libraries:** Plotly.js (interactive charts); pako (zlib inflation for compressed MATLAB v5 files); h5wasm (reading MATLAB v7.3/HDF5 files). These handle plotting and file decoding only, not the DSP.
- **Platform:** Web (single-page application); heavy computation runs in a Web Worker to keep the interface responsive on large files. Deployed on Netlify.

- **Validation tooling:** A Node.js test suite for the DSP functions and parsers; Python (scipy/h5py) used only to generate independent ground-truth files for testing the parsers.
- **IDE:** VS Code.

6 Mathematical Formulation

This section sets out, step by step, the calculations the application performs between reading a raw current file and producing a health verdict.

Synchronous speed. From supply frequency f (Hz) and pole count P :

$$N_s = 120 \times f / P$$

Slip. The fractional difference between synchronous and rated speed N_r :

$$s = (N_s - N_r) / N_s$$

Slip frequency. Slip expressed as a frequency:

$$f_{\text{slip}} = s \times f$$

Fault sideband frequencies. A mechanical fault modulates the current at twice the slip frequency, producing sidebands symmetric about the fundamental:

$$f_{\text{sideband}} = f \pm 2k f_{\text{slip}} \quad (k = 1 \text{ for the first-order sidebands used here})$$

Windowed FFT. A Hann window is applied before transforming, to control leakage:

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right), \quad X[k] = \sum_{n=0}^{N-1} x[n] w[n] e^{-j2\pi kn/N}$$

Sideband strength. For each sideband, the spectral magnitude nearest that frequency is read off and expressed relative to the fundamental, in decibels:

$$L_{\text{dB}} = 20 \log_{10}(A_{\text{sideband}} / A_{\text{fundamental}})$$

Health verdict rule. The worse (less negative) of the two sideband levels is compared against a threshold T (default -40 dB):

$$\text{verdict} = \text{FAULT LIKELY if } \max(L_{\text{dB,low}}, L_{\text{dB,high}}) > T, \text{ else HEALTHY}$$

Analytic signal and envelope, from the Hilbert transform $\mathcal{H}\{\cdot\}$:

$$x_a(t) = x(t) + j \mathcal{H}\{x(t)\}, \quad \text{envelope}(t) = |x_a(t)|$$

The envelope is then run through the same windowed-FFT step, which isolates the amplitude-modulation frequency imposed by a fault, separately from the strong 50/60 Hz carrier.

7 Work Remaining

- Run the pipeline across the full set of healthy and faulty recordings in the chosen dataset, across all provided load conditions, and record the resulting sideband strengths.
- Use that data to calibrate the fault-detection threshold properly, replacing the current literature-based -40 dB placeholder with a value that actually separates the healthy and faulty populations in this dataset.

- Check whether the same threshold generalises across load conditions, or whether load-dependent calibration is required (MCSA sideband strength is known to vary with load).
- Extract and tabulate the comparison between FFT, Welch PSD, and Hilbert-envelope results across the dataset, to fully address the objective of comparing the three spectral methods.
- Record the demo video and finalise the written report with the completed results.

None of this remaining work requires changes to the core architecture; it is testing, data analysis, and calibration layered on top of a pipeline that is already built and functioning.

8 Challenges Faced

7.1 Dataset mismatch (MAFAULDA lacks a current channel). The dataset specified in the original proposal turned out to contain only vibration and acoustic channels — no current measurement at all. This was caught by checking the dataset’s own documentation. The solution was to source a verified dataset that genuinely contains current channels.

7.2 A first replacement dataset was unusable. The first alternative logged current far too slowly to be a raw waveform — effectively a trend log, identifiable by long runs of identical consecutive values. Running the pipeline on it would have produced plausible-looking but meaningless numbers. This was caught during manual inspection, and the check was then built into the application as an automatic signal-quality warning.

7.3 Numerical instability in the bandpass filter. The filter was originally the direct transfer-function form of a fourth-order Butterworth. For a narrow passband relative to a 50 kHz sampling rate, this form loses precision and produced unstable/overflowing output. It was caught by testing the filter against a synthetic signal, and fixed by re-implementing it in second-order-sections (SOS) form, then re-verifying the output was bounded.

7.4 Multi-column file parsing. The loader initially assumed a single current column and crashed on the real files (header row plus nine columns) when it tried to convert a text header to a number. The fix detects the header and column count and prompts for which column is the current channel.

7.5 A false fault traced to an incorrect frequency assumption. During early testing, a recording labelled healthy was reported as faulty. Comparing the fundamental peak against the rest of the spectrum showed almost no content at the assumed 60 Hz — the motor actually runs at 50 Hz, so the pipeline was measuring noise. Once corrected, the file returned a healthy verdict, and a fundamental-dominance check was added to warn about this automatically.

7.6 Minor interface bug. A panel-numbering format error crashed the comparison tab when it tried to label panels with letters using an integer-only format. A straightforward fix once traced to its source.

Notably, every false result encountered so far was a data or configuration problem, not an error in the underlying DSP mathematics — a reasonable sign the core logic is sound, and precisely why the automatic sanity checks were added rather than treated as optional polish.

9 Individual Contribution

This is an individual project. All problem formulation, dataset evaluation and selection, DSP pipeline design and implementation, debugging, interface design, and deployment described in this report were carried out by the submitting student.

10 Timeline for Remaining Work

Week / Date Range	Planned Task
Week 1	Run all healthy/faulty files through the pipeline; record sideband strengths
Week 2	Calibrate the fault threshold from real data; check load dependence
Week 3	Tabulate FFT vs Welch vs envelope comparison across the dataset
Week 4	Finalise report + record demo video + viva preparation

Reference links.

- Figshare current dataset (in use): <https://doi.org/10.6084/m9.figshare.27216219>
- Associated *Scientific Data* paper: <https://www.nature.com/articles/s41597-025-05437-3>
- Live application: <https://arpitmotorsense.netlify.app/>
- MAFAULDA (evaluated, not used — vibration/acoustic only): official UFRJ MAFAULDA project page.

This is a mid-term progress report. The final report with all sections will be submitted after end-term exams.